

Animes RD — Da Ideia ao Produto Escalável

Proposta de Evolução de Infraestrutura | Maio 2026

Sobre a Plataforma

Animes RD é uma plataforma social focada em anime, construída para ser mais do que um simples catálogo. A proposta é criar um ecossistema colaborativo onde a comunidade avalia, discute, aprova conteúdo e é guiada por inteligência artificial contextual — uma experiência única no mercado.

Funcionalidades centrais da plataforma:

- Sistema de grupos e avaliações colaborativas
- Fila de aprovação de conteúdo pela comunidade
- Rankings dinâmicos atualizados em tempo real
- Gráficos, estatísticas e analytics da comunidade
- Motor de recomendações (IA — fase futura)
- Suporte nativo para web e mobile

Onde Estamos Hoje

A plataforma **existe e está funcionando**. Não é um projeto do zero — é uma aplicação ativa, com produto validado e usuários reais.

A infraestrutura atual foi construída para validação, não para escala:

O que temos hoje	Limitação real
Frontend no GitHub Pages	Sem CDN global, sem proteção DDoS, sem cache configurável, domínio com restrições
Banco de dados no plano gratuito	Sem réplicas, sem failover, limite de conexões, risco de perda de dados, downtime em manutenções
Sem camada de cache	Cada acesso consulta o banco diretamente — lento em picos de tráfego
Sem escalabilidade	Se a plataforma viralizar, o serviço cai
Sem observabilidade	Não sabemos o que acontece em produção em tempo real
Sem CI/CD estruturado	Deploy manual, sem testes automatizados, sem reversão automática

Isso não é um problema — foi uma escolha inteligente. A infraestrutura gratuita serviu para provar o produto sem custo. Agora que a ideia está validada, é hora de construir a fundação que vai suportar o crescimento real.

O Que Está em Risco Sem Evolução

Manter a infraestrutura atual enquanto a plataforma cresce cria riscos concretos:

Para os usuários: lentidão em picos, downtime sem aviso, ausência de tempo real e funcionalidades limitadas.

Para o negócio: impossibilidade de escalar para campanhas ou parcerias, sem dados de uso para decisões, risco de perda de dados, credibilidade comprometida com investidores.

Para o produto: funcionalidades como rankings em tempo real, WebSocket e app mobile são inviáveis na infraestrutura atual. E a base para IA — o grande diferencial planejado para o futuro — simplesmente não existe hoje.

A Solução — Infraestrutura AWS

A proposta é migrar a Animes RD para uma infraestrutura profissional na AWS, construída para crescer.

Não é uma reescrita do produto. É uma **evolução da fundação** — o código existente é migrado, as funcionalidades continuam, e a experiência do usuário melhora desde o primeiro dia.

Visão Geral



Antes × Depois

	Hoje	Com AWS
Frontend	GitHub Pages	S3 + CloudFront (CDN global)
Banco de dados	Plano gratuito, sem réplica	Aurora PostgreSQL Multi-AZ, failover < 30s
Cache	Nenhum	Redis ElastiCache
Escalabilidade	Zero	Auto Scaling 2 → 50 containers
Tempo real	Não suportado	WebSocket nativo
IA (futuro)	Não disponível	Infraestrutura preparada — microsserviço dedicado
Mobile	API instável	API robusta, pronta para iOS e Android
Uptime	Dependente do plano gratuito	99,99% com failover automático
Segurança	Básica	WAF, Shield, Cognito, Secrets Manager
Observabilidade	Nenhuma	CloudWatch, X-Ray, Grafana

Deploy	Manual	CI/CD com rollback automático
--------	--------	-------------------------------

Infraestrutura Detalhada

A seguir, cada componente da nova arquitetura é descrito em detalhe — o que é, como funciona e qual problema resolve para a Animes RD.

1. DNS Global — Route 53

O que é: Serviço de DNS gerenciado da AWS. É a "lista telefônica" da internet para a plataforma — traduz `animesrd.com` para o endereço do servidor correto.

Como funciona na Animes RD:

Subdomínio	Destino
<code>animesrd.com</code>	Frontend da plataforma
<code>api.animesrd.com</code>	Backend principal (API Core)
<code>ai.animesrd.com</code>	Serviço de IA (reservado — fase futura)
<code>cdn.animesrd.com</code>	Assets estáticos e mídia

Recursos ativos:

- **Health Checks contínuos:** o Route 53 testa a saúde dos servidores a cada 30 segundos. Se um servidor falhar, o tráfego é redirecionado automaticamente — o usuário nem percebe
- **Failover automático:** em caso de falha regional, o tráfego é desviado para outra região em segundos
- **Latency-Based Routing:** direciona cada usuário para o servidor geograficamente mais próximo, reduzindo latência
- **TTL configurável:** controle fino sobre o tempo de propagação de mudanças DNS

Relevância para o negócio: hoje, um problema no servidor derruba a plataforma inteira. Com Route 53, falhas são tratadas automaticamente, sem intervenção da equipe.

2. CDN e Proteção — CloudFront + WAF + Shield + ACM

2.1 CloudFront — Distribuição Global de Conteúdo

O que é: Rede de distribuição de conteúdo (CDN) com mais de 400 pontos de presença ao redor do mundo.

Como funciona: Em vez de todos os usuários acessarem o servidor principal no Brasil, o CloudFront armazena cópias do conteúdo nos servidores mais próximos de cada usuário. Um usuário em São Paulo, Tóquio ou Lisboa recebe o conteúdo do servidor mais próximo geograficamente.

O que é distribuído via CloudFront:

- Frontend completo (HTML, CSS, JavaScript)
- Imagens, capas, avatares e banners dos animes
- Respostas de API cacheáveis (rankings, tops, listas)
- Assets do futuro aplicativo mobile

Configurações de cache:

```
/static/*      → TTL 365 dias  (arquivos com hash no nome – imutáveis)
/api/rankings/* → TTL 60s      (cache com revalidação automática)
```

/api/anime/* → TTL 300s (dados que mudam pouco)
POST / PUT / DELETE → sem cache (operações de escrita nunca são cacheadas)

Impacto direto:

- Redução de latência de até 70% para usuários fora da região de origem
- Redução de custo de banda no servidor de origem em até 80%
- Capacidade de absorver picos de tráfego massivos sem sobrecarregar os servidores

2.2 AWS WAF — Firewall de Aplicação Web

O que é: Firewall que inspeciona cada requisição HTTP antes de chegar aos servidores da plataforma.

Proteções ativas:

- **Rate limiting:** máximo de 100 requisições por segundo por IP — bloqueia flood automaticamente
- **Bot mitigation:** bloqueia User-Agents conhecidos de scrapers e bots maliciosos
- **OWASP Top 10:** proteção gerenciada contra SQL Injection, XSS e outras vulnerabilidades críticas
- **Geo-blocking:** bloqueio por país configurável quando necessário
- **IP reputation lists:** bloqueia automaticamente IPs com histórico de atividade maliciosa

Relevância: hoje a plataforma não tem nenhuma proteção nesse nível. Um ataque de bot poderia gerar custos inesperados no banco de dados gratuito ou derrubar o serviço completamente.

2.3 AWS Shield — Proteção DDoS

O que é: Serviço de proteção contra ataques de negação de serviço distribuído (DDoS) — ataques que tentam derrubar o serviço enviando volumes absurdos de tráfego falso.

- **Shield Standard:** integrado nativamente ao CloudFront, sem custo adicional. Protege contra os ataques DDoS mais comuns nas camadas de rede e transporte
- **Shield Advanced:** para quando a plataforma atingir escala — inclui resposta 24/7 da equipe de resposta DDoS da AWS e proteção financeira contra custos gerados por ataques

2.4 ACM — Certificado SSL/TLS

O que é: Gerenciador de certificados HTTPS da AWS.

Emite e renova automaticamente o certificado wildcard *.animesrd.com — todos os subdomínios têm HTTPS sem custo e sem risco de expiração acidental. A renovação acontece antes do vencimento, de forma transparente.

3. Frontend — S3 + CloudFront (substitui GitHub Pages)

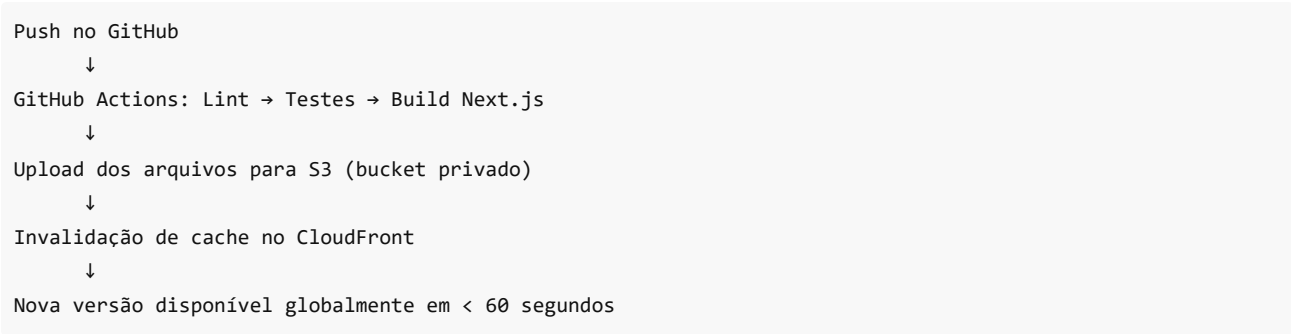
Tecnologia: Next.js (export estático) ou React SPA

Por que S3 + CloudFront é melhor que GitHub Pages:

Critério	GitHub Pages	S3 + CloudFront
CDN global	Limitado	400+ pontos de presença
HTTPS customizado	Básico	Certificado wildcard gerenciado automaticamente
Proteção DDoS	Não	AWS Shield integrado
Cache granular	Não	TTL configurável por rota
Deploy sem downtime	Não	Invalidação de cache parcial
Subdomínios ilimitados	Não	Sim
Integração com WAF	Não	Sim

Versionamento	Não	Histórico completo no S3 para rollback instantâneo
---------------	-----	--

Pipeline de deploy automatizado:



Segurança do bucket: configurado como estritamente privado. Nenhum usuário acessa o S3 diretamente — todo o tráfego passa obrigatoriamente pelo CloudFront, que aplica WAF e Shield.

4. Rede Isolada — VPC (Virtual Private Cloud)

O que é: Uma rede privada virtual dentro da AWS. O equivalente a ter um datacenter privado — isolado da internet pública, com controle total sobre o tráfego interno.

Configuração de rede:

CIDR: 10.0.0.0/16 — suporta até 65.536 endereços internos.

Tipo	Sub-rede	AZ	CIDR	Uso
Pública	animesrd-public-1a	us-east-1a	10.0.1.0/24	ALB, IGW, NAT
Pública	animesrd-public-1b	us-east-1b	10.0.2.0/24	ALB, IGW, NAT
Pública	animesrd-public-1c	us-east-1c	10.0.3.0/24	ALB, IGW, NAT
Privada	animesrd-private-1a	us-east-1a	10.0.11.0/24	ECS, Aurora, Redis
Privada	animesrd-private-1b	us-east-1b	10.0.12.0/24	ECS, Aurora, Redis
Privada	animesrd-private-1c	us-east-1c	10.0.13.0/24	ECS, Aurora, Redis

Internet Gateway: único ponto de entrada de tráfego legítimo vindo da internet. Conectado ao ALB nas subnets públicas.

NAT Gateway: permite que os servidores de backend façam requisições de saída (ex: serviços externos, webhooks, integrações) sem expor seus IPs diretamente à internet. Um NAT Gateway por AZ elimina ponto único de falha no tráfego de saída.

Regra fundamental: servidores de aplicação, banco de dados e cache ficam nas subnets privadas e **nunca são acessíveis diretamente pela internet**. Apenas o Load Balancer, nas subnets públicas, é exposto.

5. Balanceamento de Carga — Application Load Balancer

O que é: Distribui o tráfego de entrada entre os servidores disponíveis. Garante que nenhum servidor fique sobrecarregado e que falhas individuais não impactem os usuários.

Roteamento inteligente por path:

Rota	Serviço de destino
/api/*	API Core Service (Node.js)

/ai/*	IA Service (reservado — fase futura)
/ws/*	WebSocket via Socket.IO

Recursos ativos:

- **SSL Termination:** descriptografa HTTPS uma vez e distribui internamente em HTTP, reduzindo carga computacional nos servidores
- **Health Checks:** testa cada servidor a cada 30 segundos. Servidores com problema são removidos do pool automaticamente até se recuperarem
- **Cross-Zone Load Balancing:** distribui tráfego uniformemente entre todas as Zonas de Disponibilidade
- **Sticky Sessions desabilitado:** a aplicação é stateless — qualquer servidor pode atender qualquer requisição, graças ao estado centralizado no Redis
- **WebSocket nativo:** suporte completo ao protocolo WebSocket, essencial para a funcionalidade de tempo real

Multi-AZ: o ALB está distribuído nas três Zonas de Disponibilidade. A falha de uma AZ inteira não interrompe o balanceamento.

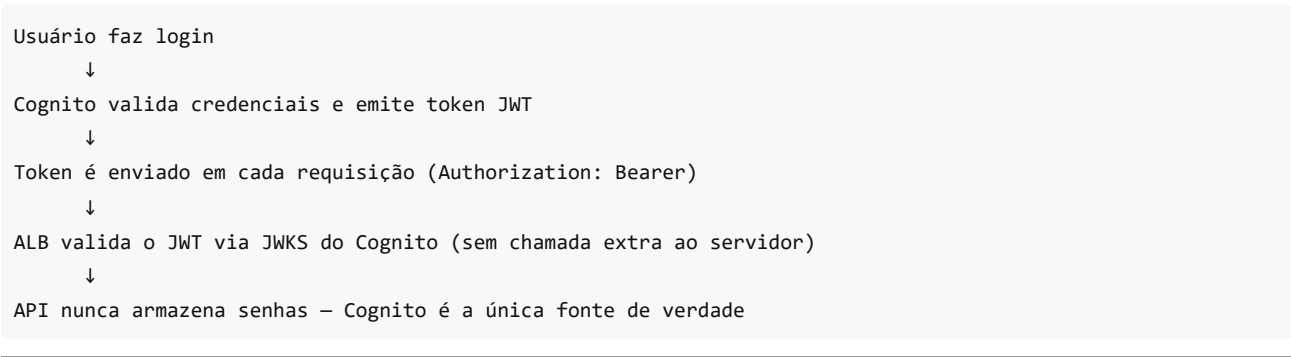
6. Autenticação — AWS Cognito

O que é: Serviço gerenciado de identidade e autenticação. Cuida de todo o ciclo de login, cadastro e autorização sem necessidade de desenvolver e manter código de segurança crítico internamente.

Funcionalidades entregues:

Funcionalidade	Disponível
Cadastro e login por e-mail e senha	Imediato
Recuperação de senha	Imediato
Tokens JWT (Access + Refresh + ID)	Imediato
Login social: Google	Configuração (sem desenvolvimento)
Login social: Discord	Configuração (sem desenvolvimento)
MFA (autenticação em dois fatores)	Ativação por toggle
Bloqueio automático por tentativas suspeitas	Nativo

Fluxo de autenticação:



7. Servidores — ECS Fargate (Auto Scaling)

O que é: Plataforma de containers serverless da AWS. Os servidores rodam em Docker, e a AWS gerencia toda a infraestrutura subjacente — sem máquinas virtuais para provisionar, sem sistema operacional para atualizar, sem capacidade para planejar manualmente.

Escalabilidade automática:

Situação	Containers por serviço
Horário de baixo uso (madrugada)	2
Tráfego normal	5 a 10
Pico de uso (horário nobre)	20 a 30
Evento viral ou campanha	até 50

Gatilhos de escala:

- CPU acima de 70% → sobe containers
- Memória acima de 80% → sobe containers
- Volume de requisições no ALB acima do threshold → sobe containers
- Fila do Worker com mais de 1.000 jobs → sobe Workers

O custo acompanha o uso real. Não há pagamento por capacidade ociosa.

7.1 API Core — Backend Principal

Tecnologia: Node.js · TypeScript · NestJS · Prisma ORM

Responsabilidades completas:

- Autenticação e autorização de usuários (integrado com Cognito)
- CRUD completo de usuários, perfis e preferências
- Sistema de grupos, membros e permissões
- Acervo de animes com metadados
- Motor de avaliações e cálculo de médias
- Sistema de comentários com threads e votos
- Fila de aprovação de conteúdo com votação
- Motor de rankings dinâmicos
- Gráficos e estatísticas da plataforma
- Perfis públicos e histórico de assistido
- WebSocket (Socket.IO) para atualizações em tempo real
- Integração futura com Serviço de IA

Arquitetura interna — Monólito Modular NestJS:

A API é estruturada em módulos independentes. Cada módulo pode ser extraído como microserviço separado no futuro sem reescrita:

AuthModule	→ JWT, OAuth, integração Cognito
UsersModule	→ Perfis, preferências, histórico
GroupsModule	→ Grupos, membros, permissões
AnimeModule	→ Acervo, CRUD, metadados, gêneros
RatingsModule	→ Avaliações, médias ponderadas, trending
CommentsModule	→ Comentários, threads, votos
ApprovalModule	→ Fila de aprovação, sistema de votação
RankingsModule	→ Motor de rankings dinâmicos
ChartsModule	→ Gráficos, análises visuais, exportações
AnalyticsModule	→ Eventos de uso, métricas internas
NotificationsModule	→ WebSocket, eventos em tempo real

Por que NestJS: framework TypeScript com injeção de dependências, decorators e estrutura modular que facilita testes automatizados, manutenção e o eventual desmembramento em microserviços.

7.2 Serviço de IA — Visão de Futuro

A arquitetura foi projetada para suportar um microserviço de IA dedicado em uma fase futura. A infraestrutura atual — Aurora PostgreSQL com suporte a pgvector, Redis para cache de respostas e ECS Fargate com Auto Scaling independente — já está preparada para receber esse serviço sem nenhuma mudança estrutural.

Quando chegar o momento, as funcionalidades planejadas incluem recomendações personalizadas por perfil de gosto, agente contextual alimentado pelos dados reais da comunidade, análise de controvérsia e busca semântica no acervo. Essa é uma das apostas mais fortes da plataforma como diferencial de mercado a longo prazo.

7.3 Worker Service — Processamento Assíncrono

Tecnologia: Node.js · TypeScript · BullMQ · Redis Queue

Por que existe: tarefas pesadas não devem bloquear a API principal. O Worker processa jobs em background, garantindo que a experiência do usuário nunca seja impactada por processamentos demorados.

Jobs processados:

Job	Gatilho	Prioridade
Recalcular rankings globais	Nova avaliação submetida	Alta
Recalcular score de controvérsia	Acúmulo de votos divergentes	Média
Atualizar perfil de gosto do usuário	10+ interações acumuladas	Baixa
Enviar notificações	Evento de fila	Alta
Exportar dados do usuário	Solicitação explícita	Baixa
Limpeza de cache expirado	Scheduler diário	Baixa
Processar uploads de imagens	Arquivo enviado ao S3	Média

Escalabilidade independente: o Worker escala baseado na profundidade da fila. Em eventos com muitas avaliações simultâneas (ex: lançamento de anime popular), mais Workers são levantados automaticamente para processar o backlog.

8. Banco de Dados — Aurora PostgreSQL (substitui o plano gratuito)

O que é: Banco de dados relacional gerenciado pela AWS, compatível com PostgreSQL. Combina a familiaridade do Postgres com alta disponibilidade, escalabilidade e performance superiores ao PostgreSQL padrão.

Diferença fundamental em relação ao plano gratuito atual:

	Plano gratuito atual	Aurora PostgreSQL
Réplicas de leitura	Nenhuma	2 réplicas automáticas
Failover automático	Não existe	< 30 segundos
Backup automático	Não garantido	Diário para S3, retenção 30 dias
Manutenção com downtime	Sim, sem aviso	Zero downtime
Limite de conexões	Baixo e fixo	Gerenciado via RDS Proxy
pgvector (busca semântica)	Não disponível	Disponível — preparado para IA futura
Monitoramento	Nenhum	Performance Insights integrado

Topologia Multi-AZ:

```
us-east-1a → Instância Primária (leitura + escrita)
              ↓ replicação síncrona
us-east-1b → Réplica Standby (assume em < 30s se primária falhar)
              ↓ replicação assíncrona
us-east-1c → Réplica de Leitura (absorve queries de relatório e analytics)
```

O endpoint do cluster nunca muda. A aplicação continua conectando no mesmo endereço, independentemente de qual instância está ativa.

Endpoints separados:

Endpoint	Uso
cluster.rds.amazonaws.com (Writer)	Todas as operações de escrita
cluster.ro.rds.amazonaws.com (Reader)	Consultas pesadas, relatórios, analytics

RDS Proxy: camada de pooling de conexões entre ECS e Aurora. Quando o ECS escala para 50 containers, cada um tentando abrir conexões com o banco poderia causar sobrecarga. O RDS Proxy gerencia um pool eficiente, reutilizando conexões existentes. Essencial para o Auto Scaling funcionar sem degradar o banco.

pgvector — preparação para IA futura:

O Aurora é provisionado com suporte à extensão pgvector, que permite busca por similaridade semântica. Quando o serviço de IA for desenvolvido na fase seguinte, o banco de dados já estará pronto para suportá-lo sem nenhuma migração ou reestruturação.

Estrutura de dados principal:

Categoria	Tabelas
Identidade	users, user_profiles, user_preferences
Social	groups, group_members, friendships
Conteúdo	anime, genres, anime_genres, links
Avaliações	ratings, comments, comment_votes
Aprovação	approval_queue, approval_votes
Status	watch_status, watch_history
Rankings	ranking_snapshots, controversy_scores, trending_scores

Backups e retenção:

Ambiente	Backup	Retenção
Produção	Automático diário + Point-in-Time Recovery	30 dias
Staging	Automático diário	7 dias
Dev	Automático diário	3 dias

9. Cache — ElastiCache Redis

O que é: Armazenamento de dados em memória RAM, extremamente rápido. Serve como camada intermediária entre a aplicação e o banco de dados.

Por que é essencial: sem cache, cada requisição de ranking ou dashboard consulta o banco de dados — operações que envolvem joins complexos e podem levar centenas de milissegundos. Com Redis, a mesma resposta é entregue em menos de 1ms.

Impacto em números: redução de até 90% das consultas ao banco de dados em horários de pico.

Dados armazenados e TTL:

Chave Redis	Conteúdo	Validade
rankings:top100	Top 100 animes globais	60 segundos
rankings:genre:{id}	Top animes por gênero	2 minutos
user:session:{id}	Sessão JWT do usuário	24 horas
user:dashboard:{id}	Dashboard personalizado	5 minutos
controversy:top50	Animes mais controversos	2 minutos
group:stats:{id}	Estatísticas de grupo	3 minutos
chart:data:{type}:{period}	Dados de gráficos	90 segundos
ws:online:{userId}	Status de presença online	30 segundos (heartbeat)

Uso como fila (BullMQ):

O Redis também serve como broker de mensagens para o sistema de filas. Os Workers consomem jobs destas filas:

```
bullmq:rankings:waiting    → jobs aguardando processamento
bullmq:rankings:active     → jobs em execução
bullmq:rankings:completed → histórico de execução
bullmq:rankings:failed     → jobs com erro (para reprocessamento)
```

Configuração Alta Disponibilidade: Replication Group Multi-AZ com failover automático. Uma instância Redis ativa e uma réplica em standby em outra AZ.

10. Armazenamento de Arquivos — Amazon S3

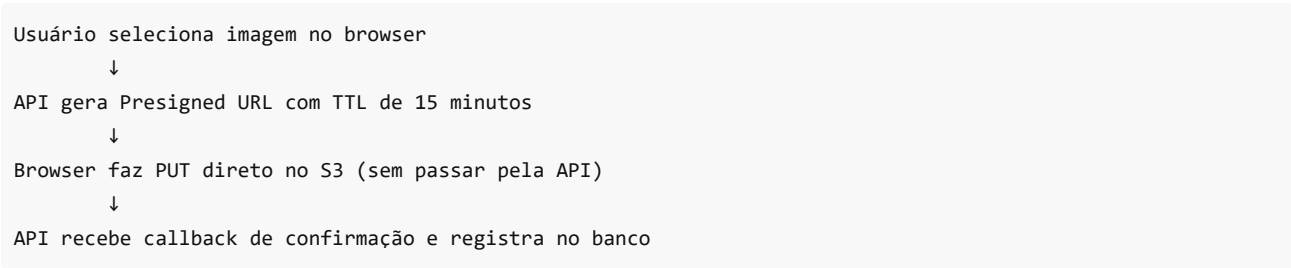
O que é: Serviço de armazenamento de objetos com disponibilidade de 99,999999999% (onze noves) e durabilidade garantida.

Estrutura de buckets:

Bucket	Conteúdo	Acesso
animesrd-assets-prod	Avatares, capas, banners	Público via CloudFront (OAC)
animesrd-uploads-prod	Uploads temporários dos usuários	Presigned URL (TTL 15 min)
animesrd-exports-prod	Exports de dados gerados	Presigned URL (TTL 1 hora)
animesrd-backups-prod	Backups do Aurora, logs de auditoria	Privado — acesso só via IAM Role
animesrd-frontend-prod	Build Next.js compilado	Privado via CloudFront (OAC)

Upload direto pelo browser:

O usuário envia arquivos (avatars, imagens) diretamente para o S3 usando URLs pré-assinadas geradas pela API. O arquivo **nunca passa pelo servidor backend** — reduz carga, acelera upload e elimina limitações de tamanho no servidor.



Ciclo de vida: imagens de upload temporário são automaticamente excluídas após 24 horas se não confirmadas. Exportações são excluídas após 7 dias.

11. Tempo Real — WebSocket

O que é: Protocolo de comunicação bidirecional persistente. Permite que o servidor envie atualizações para o browser sem que o usuário precise recarregar a página ou fazer novas requisições.

Implementação: Socket.IO rodando no API Core, com estado compartilhado via Redis usando o adapter `@socket.io/redis-adapter`. Isso permite que múltiplos containers ECS atendam o mesmo usuário de forma consistente.

Eventos transmitidos em tempo real:

Evento	Experiência para o usuário
anime:rating-updated	A nota do anime se atualiza na tela enquanto outros avaliam
comment:new	Novo comentário aparece instantaneamente, sem reload
approval:vote-cast	Placar da fila de aprovação se move em tempo real
ranking:updated	O Top animes reflete mudanças imediatamente
user:online-status	Presença de membros do grupo visível em tempo real
notification:new	Notificação aparece sem necessidade de checar

12. Segurança — Gestão de Credenciais

AWS Secrets Manager

Todas as credenciais sensíveis da plataforma são armazenadas no Secrets Manager — nunca em código-fonte, repositórios Git ou variáveis de ambiente expostas.

Credenciais gerenciadas:

Secret	Serviço que usa
/animesrd/prod/database/url	API Core, Workers
/animesrd/prod/redis/url	API Core, Workers
/animesrd/prod/jwt/secret	API Core
/animesrd/prod/cognito/client-secret	API Core

Rotação automática: credenciais do banco de dados são rotacionadas automaticamente a cada 30 dias, sem downtime — o Aurora e o Secrets Manager coordenam a troca sem impacto na aplicação.

Princípio de menor privilégio: cada serviço ECS tem uma IAM Role com permissão de acesso apenas aos secrets que realmente precisa — um comprometimento em um serviço não expõe as credenciais dos demais.

13. Observabilidade Completa

Hoje a Animes RD opera no escuro — problemas são descobertos quando usuários reclamam. A nova infraestrutura entrega visibilidade total.

AWS CloudWatch

Centraliza logs, métricas e alertas de todos os componentes da plataforma.

O que é monitorado automaticamente:

Métrica	Alerta configurado
Taxa de erros HTTP 5xx	Alerta se > 1% das requisições
Latência média da API	Alerta se p95 > 2 segundos
Uso de CPU dos containers	Alerta se > 85% por mais de 5 min
Profundidade da fila BullMQ	Alerta se > 1.000 jobs pendentes
Conexões ao banco de dados	Alerta se > 80% do pool
Conexões WebSocket ativas	Métrica de engajamento em tempo real
Cache hit ratio do Redis	Alerta se < 70% (indica problema de cache)

AWS X-Ray — Distributed Tracing

Rastreia o caminho completo de cada requisição: do browser → ALB → API Core → Aurora → Redis → resposta.

O que resolve: quando um usuário reclama de lentidão, é possível visualizar exatamente em qual etapa o tempo foi gasto — banco de dados, cache, rede ou processamento interno. Sem X-Ray, esse diagnóstico é chute.

Grafana — Dashboards de Negócio

Dashboards visuais construídos sobre os dados do CloudWatch, com foco em métricas que importam para o produto:

- Usuários online por hora e por dia
 - Número de avaliações e comentários em tempo real
 - Animes em tendência no momento
 - Volumetria da fila de aprovação
 - Taxa de retenção por funcionalidade
-

14. CI/CD — Pipeline de Deploy Contínuo

Fluxo completo automatizado:



Etapa 2 – Build:

Docker build da imagem (API Core / Worker)
Tagging com SHA do commit para rastreabilidade



Etapa 3 – Publicação:

Push da imagem para Amazon ECR



Etapa 4 – Deploy:

ECS Rolling Update: sobe novos containers gradualmente
Health Check: valida saúde antes de remover os antigos
Se falhar → rollback automático para versão anterior



Etapa 5 – Frontend (em paralelo):

Build Next.js
Upload para S3
Invalidação de cache no CloudFront



Nova versão em produção sem downtime, em < 10 minutos

Amazon ECR: registro privado de imagens Docker. Cada deploy gera uma nova imagem com tag versionada. Rollback é executar a imagem de qualquer commit anterior.

Zero downtime: o ECS mantém os containers da versão atual ativos até os novos estarem saudáveis. O usuário nunca vê erro durante um deploy.

15. Mobile — Suporte Nativo

Nenhuma alteração de backend é necessária para lançar o aplicativo iOS e Android.

Os aplicativos móveis consomem exatamente as mesmas APIs REST e WebSocket que o frontend web. A autenticação é via Cognito SDK mobile nativo, com suporte a biometria (Face ID, impressão digital) sem desenvolvimento adicional.

O CloudFront otimiza automaticamente a entrega para conexões móveis, comprimindo assets e servindo do ponto de presença mais próximo do dispositivo.

16. Analytics — Pipeline de Dados (Fase Futura)

A infraestrutura atual já está preparada para um pipeline de analytics em tempo real quando a plataforma atingir escala.

Arquitetura do pipeline:

API Core coleta eventos → Kinesis Data Streams (ingestão em tempo real)



Kinesis Firehose (batching automático)



S3 Data Lake (armazenamento em Parquet)



Amazon Athena (consultas SQL serverless)



Amazon QuickSight (dashboards BI)

Eventos capturados:

anime Rated	→ nota dada a um anime
comment Added	→ comentário publicado
approval Vote	→ voto na fila de aprovação
anime Added	→ novo anime no acervo

recommendation_click	→ usuário seguiu uma recomendação
group_joined	→ entrada em um grupo
user_registered	→ novo cadastro
anime_searched	→ busca realizada

Valor para o negócio: dados estruturados de comportamento de usuário habilitam decisões de produto baseadas em evidências, relatórios para parceiros e modelos de monetização.

17. Estratégia de Ambientes

Ambiente	Propósito	Configuração
Development	Desenvolvimento de features, testes locais	Aurora Serverless v2 (escala a zero), Redis single-node — custo próximo a zero
Staging	Homologação antes de produção, demos para stakeholders	Espelho da produção em escala reduzida
Production	Plataforma ao vivo para usuários reais	Multi-AZ completo, Auto Scaling, todos os alarmes ativos

Cada ambiente tem VPC, banco, Redis, Cognito User Pool e secrets completamente isolados.

Roadmap de Evolução

Fase	Estrutura	Gatilho
Hoje	GitHub Pages + banco gratuito	Estado atual
Migração	Monólito Modular na AWS (infraestrutura preparada para IA)	Decisão de investimento
Crescimento	Auth Service extraído como microserviço	> 10.000 usuários ativos
Escala	Anime, Ratings, Comments Services separados	Múltiplos times de desenvolvimento
Enterprise	Analytics pipeline completo, Notification Service, API pública	Parcerias e integrações externas

Cada fase é uma evolução natural — sem parada do serviço, sem reescrita do produto.

Estimativa de Investimento

Custo Mensal de Infraestrutura (Produção)

Serviço	Estimativa mensal
CloudFront + S3 (CDN + frontend + assets)	U\$ 15 – 25
ECS Fargate (API Core + Workers)	U\$ 60 – 120
Aurora PostgreSQL Multi-AZ	U\$ 100 – 180
ElastiCache Redis (Replication Group)	U\$ 30 – 60
Application Load Balancer	U\$ 25
NAT Gateway (Multi-AZ)	U\$ 35
AWS WAF	U\$ 15

CloudWatch + X-Ray	U\$ 15 – 30
Route 53	U\$ 5
Secrets Manager + ECR + ACM	U\$ 10
Total estimado	U\$ 310 – 505 / mês

O Que Esse Investimento Entrega

Entrega	Impacto
Banco de dados que nunca cai	Confiança do usuário e do parceiro
Escala automática	Suporta qualquer campanha ou evento sem preparação manual
Base pronta para IA (fase futura)	Infraestrutura que suporta o próximo diferencial
App mobile viável	Novo canal de crescimento e retenção
Observabilidade total	Decisões de produto baseadas em dados reais
Segurança enterprise	Credibilidade em negociações comerciais e parcerias
CI/CD automatizado	Time de desenvolvimento mais rápido e confiante

Otimizações de Custo Previstas

- **Savings Plans ECS Fargate (1 ano):** redução de até 50% no custo de compute
- **Aurora Serverless em dev/staging:** escala a zero quando não utilizado — custo zero fora do horário de uso
- **Cache agressivo no CloudFront:** reduz requisições ao servidor de origem em até 80%
- **S3 Intelligent-Tiering:** assets antigos migrados automaticamente para camadas mais baratas

Checklist de Segurança

- ✓ Backend, banco de dados e cache em subnets privadas — sem exposição direta à internet
- ✓ Security Groups com menor privilégio — Aurora aceita tráfego apenas do ECS, Redis idem
- ✓ IAM Roles por serviço — cada container tem permissão mínima necessária
- ✓ Todas as credenciais no Secrets Manager — zero hardcode em código ou variáveis de ambiente
- ✓ Criptografia em repouso: Aurora (KMS), Redis, S3
- ✓ Criptografia em trânsito: HTTPS em todas as camadas (ACM)
- ✓ WAF com proteção OWASP Top 10 e rate limiting
- ✓ Buckets S3 privados — acesso exclusivo via CloudFront OAC
- ✓ CloudTrail habilitado — auditoria completa de todas as ações na AWS
- ✓ GuardDuty — detecção de ameaças e comportamentos suspeitos
- ✓ Rotação automática de credenciais do banco de dados a cada 30 dias

Resumo da Proposta

A Animes RD tem produto validado, comunidade ativa e diferenciais claros de mercado. O que falta é a fundação tecnológica que permita crescer sem quebrar.

A migração para AWS transforma a plataforma de um projeto de validação em um produto pronto para escala, parceria e monetização.

	Situação atual	Com esta arquitetura
--	----------------	----------------------

Suporta crescimento repentino	Não — servidor único cai	Sim — Auto Scaling automático
Pode lançar app mobile	Com risco de instabilidade	Sim — API robusta e preparada
Pronto para IA (fase futura)	Não — sem infraestrutura base	Sim — pgvector e ECS preparados
Tem dados para decisões de produto	Não	Sim — observabilidade e analytics
Pode apresentar a investidores e parceiros	Com ressalvas técnicas sérias	Sim — infraestrutura enterprise
Risco de perda de dados	Alto — banco gratuito sem backup	Mínimo — backups e réplicas automáticas
Custo operacional	R\$ 0 (com limitações críticas)	~US\$ 400/mês (infraestrutura profissional)